Package 'stratsampling'

January 30, 2026

Title Implementation of the stratified sampling for coding

Version 0.0.0.9000

Description This code implements the methods described in the stratified coding paper of Mozer, Pashley and Miratrix. It also has the code to run the simulations for that paper.

License MIT + file LICENSE

Encoding UTF-8

Roxygen list(markdown = TRUE)

Depends tidyverse

RoxygenNote 7.3.3

Suggests testthat (>= 3.0.0)

Config/testthat/edition 3

Contents

calc_nominal_var . . . 2
calc_optimal_allocation_oracle . . . 2
calc_sampling_weights . . . 3
compare_decomposition_oracle . . . 3
compare_oracle_vs_precoding . . . 5
compare_stratifications . . . 6
compare_stratifications_precoding . . . 6
compute_variance_decomposition . . . 7
create_stratification_candidates . . . 8
describe_population . . . 9
estimate_R2 . . . 10
est_ma_srs . . . 10
est_ma_strat . . . 11
est_oracle . . . 12
est_subset . . . 12
evaluate_stratification . . . 13
evaluate_stratification_precoding . . . 14
generate_observed_sample . . . 15
generate_population . . . 15
generate_sample_index . . . 16
make_power_table . . . 17
plot_decomposition_comparison . . . 18
plot_strata_residuals . . . 18

Index **19**

`calc_nominal_var` *Estimate what the SE would be given a stratification design*

Description

This method calculates the SE given some possible population summary statistics for the proportional stratified estimator. This method is used for making power calculations when planning to subsample documents.

Usage

```
calc_nominal_var(N, p, h, var_Y0, R2_ML, R2_strat)
```

Arguments

`N`	The total population size
`p`	The proportion of units assigned to treatment
`h`	The proportion of units that will be human coded (i.e., the sampling fraction)
`var_Y0`	The variance of the outcome in the control group
`R2_ML`	The pseudo R-squared of the model's surrogate outcomes for predicting actual outcomes.
`R2_strat`	The proportion of the residual variance explained by the stratification variable.

Value

The estimated variance of the model-assisted proportional stratified estimator.

`calc_optimal_allocation_oracle`
This method assumes we have Yobs known for everyone, and thus is an oracle. It will look at the actual residuals (Yobs - Yhat) to determine optimal allocation of samples across strata (and treatment arms if Z passed) to minimize variance of the model-assisted estimator.

Description

This method assumes we have Yobs known for everyone, and thus is an oracle. It will look at the actual residuals (Yobs - Yhat) to determine optimal allocation of samples across strata (and treatment arms if Z passed) to minimize variance of the model-assisted estimator.

Usage

```
calc_optimal_allocation_oracle(
  dat,
  n_code,
  Z = NULL,
  strata = NULL,
  Yobs = "Yobs",
  Yhat = "Yhat",
  return_counts = FALSE,
  seed = NULL
)
```

---

calc_sampling_weights *Given a sampling indicator and stratification, calculate the associated sampling weights.*

Description

Given a sampling indicator and stratification, calculate the associated sampling weights.

Usage

```
calc_sampling_weights(dat, Z = NULL, S = "S", strata = NULL)
```

Arguments

S	Name of column in dat that is the sampling indicator (0/1). Or vector of values.

Value

Vector of sampling probabilities (weights) for each unit in dat.

---

compare_decomposition_oracle *Compare Candidate Stratifications Using Oracle Variance Decomposition*

Description

For each candidate stratification, computes the BS (between-strata) and WS (within-strata) variance components from Theorem 3.2, which together determine the variance reduction from stratified sampling. Returns a summary table and visualization comparing candidates based on oracle information (true residuals).

Usage

```
compare_decomposition_oracle(data, candidates, n_code, Z = "Z")
```

Arguments

`data`	Data frame containing `Yobs`, `Yhat`, and optionally a treatment indicator column.
`candidates`	Named list of candidate stratification vectors. Each element should be a factor or character vector of the same length as `nrow(data)` containing stratum assignments.
`n_code`	Total number of units to sample for human coding. Required for computing the exact Theorem 3.2 formulas.
`Z`	Character string specifying the name of the treatment indicator column, or `NULL` to ignore treatment groups. Default is `"Z"`.

Details

This function uses oracle information (the true residuals Yobs - Yhat) to compute the exact variance decomposition for each candidate stratification. The key metric is BS - WS, which represents the variance reduction achieved by stratified sampling compared to simple random sampling (Theorem 3.2):

$$Var(\hat{\tau}_{SRS}) - Var(\hat{\tau}_{strat}) = BS - WS$$

Stratifications with larger BS - WS values provide greater variance reduction. The BS component measures systematic differences in prediction bias across strata, while the WS component (typically small under proportional allocation) reflects within-stratum residual heterogeneity.

Value

A list with three elements:

`summary` Data frame with one row per stratification containing:

- `stratification`: Name of the stratification
- `n_strata`: Number of strata
- `BS`: Between-strata variance component
- `WS`: Within-strata variance component
- `BS_minus_WS`: Variance reduction (BS - WS)
- `var_ratio`: Ratio of max to min within-stratum variance
- `rank`: Rank by BS_minus_WS (1 = best)

`decomposition_details` List of full decomposition results for each stratification (output from `compute_variance_decomposition`).

See Also

`compute_variance_decomposition` for the underlying decomposition.

`compare_oracle_vs_precoding` *Compare Pre-coding Proxy Metrics with Oracle Metrics*

Description

Creates a combined table and visualization comparing the pre-coding proxy metrics (based only on Yhat) with oracle metrics (based on actual residuals) for multiple candidate stratifications. This validates whether the pre-coding metrics successfully identify stratifications that perform well in practice.

Usage

```
compare_oracle_vs_precoding(
  oracle_comparison,
  precoding_comparison,
  top_n = 10
)
```

Arguments

`oracle_comparison`	Output from `compare_stratifications`, which uses actual residuals (Yobs - Yhat).
`precoding_comparison`	Output from `compare_stratifications_precoding`, which uses only Yhat as a proxy.
`top_n`	Integer specifying how many top stratifications to include in the output. Default is 10.

Details

This function is useful for validating that the pre-coding proxy metrics (which can be computed before any human coding is done) successfully identify stratifications that would perform well according to the oracle metrics (which require knowing the true residuals).

The key comparison is between:

- `mean_spread` (oracle): Variance of mean residuals across strata (BS component)
- `mean_spread` (pre-coding): Variance of mean Yhat across strata (proxy for BS)

High correlation between rankings suggests the pre-coding metrics are reliable for selecting stratifications in practice.

Value

A list with three elements:

`combined_table` Data frame merging oracle and pre-coding metrics for each stratification, with columns for both metric types and rank comparisons.

`correlation` Spearman correlation between oracle and pre-coding rankings, measuring how well pre-coding metrics predict oracle performance.

`plot` A ggplot object showing the relationship between pre-coding and oracle metrics.

See Also

compare_stratifications, compare_stratifications_precoding

`compare_stratifications` *Compare Multiple Candidate Stratification Variables*

Description

Evaluates and compares multiple candidate stratification variables to help select the best one for stratified model-assisted estimation. Each candidate is assessed using `evaluate_stratification`, and results are summarized in a ranked table.

Usage

```
compare_stratifications(data, candidates, Z = "Z", ...)
```

Arguments

`data`	Data frame containing a `resid` column (residuals = Yobs - Yhat), a treatment indicator column named `"Z"`, and any columns referenced by the candidate stratifications.
`candidates`	Named list of candidate stratification vectors. Each element should be a vector of the same length as `nrow(data)` containing stratum assignments. Names are used to identify stratifications in the output.
`min_var_threshold`	Numeric. Minimum within-stratum residual variance required. Stratifications with any stratum below this threshold are excluded from results. Default is `0.01`.

Value

A list of performance metrics for each stratification method

`compare_stratifications_precoding` *Compare candidate stratifications using pre-coding metrics*

Description

Compare candidate stratifications using pre-coding metrics

Usage

```
compare_stratifications_precoding(data, candidates, Z = NULL)
```

Arguments

data	Data frame with Yhat and Z columns
candidates	Named list of stratification vectors
Z	Character string specifying the name of the treatment indicator or NULL to ignore treatment groups.

Value

Data frame comparing stratifications

---

compute_variance_decomposition

Compute variance decomposition (BS and WS components)

Usage

```
compute_variance_decomposition(dat, Z = "Z", strata = NULL, n_code = NULL)
```

Arguments

dat	Data frame with Yobs, Yhat columns
Z	Character string specifying the treatment indicator column name, or NULL to ignore treatment groups. Default is "Z".
strata	Stratification variable. Can be a vector of stratum assignments (same length as nrow(dat)) or NULL if dat already contains a column named "strata".
n_code	Optional. Total number of units to be sampled for coding. If provided, the function computes the exact BS and WS terms from Theorem 3.2 which depend on the sampling fraction. If NULL (default), computes simplified population-level variance decomposition components that serve as proxies.

Value

If Z is specified (treatment/control), returns a list with:

by_arm Data frame with decomposition for each treatment arm

overall Data frame with overall decomposition (summed over arms)

If Z is NULL, returns a single data frame with the decomposition.

The decomposition includes:

BS Between-strata variance component. Measures how much stratum mean residuals differ from the grand mean. Larger values indicate greater potential benefit from stratification.

WS Within-strata variance component. Weighted average of within-stratum residual variances. Under proportional allocation, this term is typically small and positive.

```
  NOTE: The exact WS term from Theorem 3.2 can be negative under optimal
  allocation (oversampling high-variance strata), but the simplified
  population version computed here is always positive.}
\item{\code{BS_minus_WS}}{The difference BS - WS, which approximates
  the variance reduction from stratified sampling vs. SRS. Positive values
  indicate stratification reduces variance.}
```

Computes the between-strata (BS) and within-strata (WS) variance components of the residuals. These components are related to the variance reduction from stratified sampling as described in Theorem 3.2 of the manuscript.

`evaluate_stratification` for computing stratification quality metrics, `plot_strata_residuals` for visualization.

`create_stratification_candidates`
Create Candidate Stratification Variables

Description

Generates a set of candidate stratification variables for use in stratified model-assisted estimation. Creates single-variable stratifications (tertiles, quartiles, quintiles) for each specified variable, as well as interaction stratifications for all pairs of variables.

Usage

```
create_stratification_candidates(
  dat,
  vars,
  Z = NULL,
  jitter_vars = NULL,
  sd_jitter = 0.001,
  seed = NULL
)
```

Arguments

`dat`	Data frame containing the variables to stratify on.
`vars`	Character vector specifying the names of columns in `dat` to use for creating stratifications.
`Z`	Optional character string specifying the name of the treatment indicator variable in `dat`. If provided, stratifications are computed separately within each treatment arm, ensuring balanced strata across arms. Default is `NULL` (pooled stratification).
`jitter_vars`	Character vector specifying which variables (subset of `vars`) should have small random noise added before computing quantile breaks. This helps avoid ties that can cause `cut()` to fail. Default is `NULL` (no jitter applied).
`sd_jitter`	Numeric value specifying the standard deviation of the jitter noise. Default is `1e-3`.
`seed`	Optional integer seed for reproducibility when jitter is applied. Default is `NULL`.

Details

For each variable in `vars`, the function creates three stratifications based on quantiles:

- Tertiles (3 groups): Low, Med, High
- Quartiles (4 groups): Low, Med-Low, Med-High, High

- Quintiles (5 groups): Very Low, Low, Med, High, Very High

For each unique pair of variables, the function creates four interaction stratifications by crossing quantile groups:

- 2x2: median split on both variables (4 strata)
- 2x3: median split on first, tertiles on second (6 strata)
- 3x2: tertiles on first, median split on second (6 strata)
- 3x3: tertiles on both variables (9 strata)

When `Z` is provided, quantile breaks are computed separately within each treatment arm. This ensures that the same proportion of treated and control units fall into each stratum level, which is important for maintaining balance in stratified estimation.

When a variable has many tied values, quantile breaks may not be unique, causing `cut()` to fail. Use `jitter_vars` to add small random noise (SD = 0.001) to such variables before computing breaks.

Value

A named list of factor vectors, each representing a candidate stratification. Names follow the convention:

Single-variable `{var}_tertiles`, `{var}_quartiles`, `{var}_quintiles`

Interactions `{var1}_{var2}_2x2`, `{var1}_{var2}_2x3`, `{var1}_{var2}_3x2`, `{var1}_{var2}_3x3`

Factor levels have meaningful labels (e.g., "Low Yhat", "Med Yhat / High word_count"). Stratifications that fail due to non-unique quantile breaks are automatically excluded from the output.

`describe_population` *Describe the DGP population defined by set parameters*

Description

Utility function to explore the different simulation factors

Usage

```
describe_population(
  n_strat,
  strata_config = c("balanced-exact", "balanced-approx", "unbalanced"),
  tau = 0,
  bias = c("none", "small", "moderate", "large", "extreme-contrast"),
  residual_var = c("homogeneous", "heterogeneous", "extreme-contrast"),
  sd_y = 3,
  R2_target = 0.5,
  no_tx = FALSE,
  rescale_v = TRUE,
  seed = NULL,
  verbose = FALSE
)
```

estimate_R2 *Given data, estimate the R2 of the stratification and ML*

Description

This can be used on pilot data to get a rough sense of values for a power calculation.

Usage

```
estimate_R2(
  data = NULL,
  Yhat = "Yhat",
  Yobs = "Yobs",
  S = NULL,
  Z = "Z",
  strata = "strata"
)
```

Details

This is a back of the envelope calculation that does not take varying proportions of human coded outcomes within strata into account.

Value

A tibble containing the estimated R2 of the stratification (how much machine predictions vary across strata) and of the surrogates for the outcome.

See Also

`calc_nominal_var()`, `plot_power_curve()`, `make_power_table()`

est_ma_srs *Calculate treatment impacts with SRS sample*

Description

(Baseline) function to calculate tau_hat_ma (the model-assisted treatment effect estimator) assuming we have human-coded scores, Yobs, for a subsample of n<N documents plus predicted scores for all N documents, with the subsample chosen using SRS

Usage

```
est_ma_srs(
  data,
  Yhat = "Yhat",
  Yobs = "Yobs",
  S = "S",
  Z = "Z",
  finite_pop = FALSE
)
```

`est_ma_strat` *Calculate treatment impacts with stratified sample*

Description

Function to calculate the model-assisted treatment effect estimator assuming we have K fixed strata and hand-coded scores for a subsample of n<N documents chosen by stratified random sampling (with fixed and known sampling probabilities).

Usage

```
est_ma_strat(
  data = NULL,
  Yhat = "Yhat",
  Yobs = "Yobs",
  S = "S",
  Z = "Z",
  strata = "strata",
  return_pieces = FALSE,
  finite_pop = FALSE
)
```

Details

If strata set to null, then do the SRS version.

Value

A data.frame with columns:

`est`	the point estimate
`SE`	Not sure what this is.
`SEhat`	the estimated standard error of the estimator
`SEhat.full`	the estimated standard error component due to full coding variance (i.e., if we had coded everything)
`SEhat.samp`	the estimated standard error component due to sampling variance (i.e., due to only coding a subset of documents) HOW DIFFERENT FROM SEhat?
`S2.0`	the estimated population variance of the outcomes in the control group
`S2.1`	the estimated population variance of the outcomes in the treatment group

est_oracle *Compute oracle estimate (full coding) using shared functions*

Description

Compute oracle estimate (full coding) using shared functions

Usage

```
est_oracle(data, Yobs = "Yobs", Z = "Z")
```

Value

Data frame with oracle estimate and SE

est_subset *Simple subset coding estimator*

Description

This is the (Baseline) function to calculate tau_hat_subset (the simple subset treatment effect estimator) assuming we have human-coded scores, Yobs, for a subsample of n<N chosen using SRS

Usage

```
est_subset(data, Yobs = "Yobs", S = "S", Z = "Z", finite_pop = FALSE)
```

Value

A data.frame with columns:

est	the point estimate
SE	The SE of the estimator if everything were coded
SEhat	the estimated standard error of the estimator
SEhat.full	NA (not applicable for this estimator)
SEhat.samp	NA (not applicable for this estimator)
S2.0	the sample variance of the coded outcomes in the control group
S2.1	the sample variance of the coded outcomes in the treatment group

`evaluate_stratification` *Evaluate a Candidate Stratification Variable*

Description

Computes stratum-level residual statistics and heterogeneity metrics to assess the potential benefit of a stratification for model-assisted estimation.

Usage

```
evaluate_stratification(data, strata_var, Z = "Z", min_var_threshold = 0.01)
```

Arguments

`data`	Data frame containing a `resid` column (residuals = Yobs - Yhat) and the stratification variable specified by `strata`.
`strata_var`	Character string specifying the name of the column to use as the stratification variable.
`Z`	Character string specifying the name of the treatment indicator or `NULL` to ignore treatment groups.
`min_var_threshold`	Numeric. Minimum within-stratum residual variance required. If any stratum has variance below this threshold, the function returns `NULL`. This guards against strata with near-zero variance that could cause numerical issues. Default is `0.01`.

Details

This function is useful for comparing candidate stratification variables before selecting one for stratified sampling. Key metrics to examine:

- `var_mean_resid`: Higher values indicate greater between-strata variance (BS), suggesting stratification will reduce sampling variance.
- `ratio_var`: Values greater than 2 indicate heteroskedasticity across strata, suggesting optimal (Neyman) allocation will outperform proportional allocation.
- `imbalance`: Values close to 1 indicate balanced strata; highly unbalanced strata may reduce efficiency.

Value

A list with three elements, or `NULL` if the stratification is invalid (e.g., empty data, missing values, or variance below threshold):

`strata_stats` Data frame with stratum-level statistics:

- `N_zk` (or `N_k`): Sample size in each stratum
- `mean_resid`: Mean residual in each stratum
- `var_resid`: Variance of residuals in each stratum
- `sd_resid`: Standard deviation of residuals in each stratum

If `Z` is specified, includes treatment indicator `Z`.

`heterogeneity` Data frame summarizing heterogeneity across strata:

- `n_strata`: Number of strata
- `min_size`, `max_size`: Range of stratum sizes
- `imbalance`: Ratio of largest to smallest stratum size
- `min_var`, `max_var`: Range of residual variances
- `ratio_var`: Ratio of max to min residual variance (heteroskedasticity measure)
- `range_var`: Difference between max and min residual variance
- `var_mean_resid`: Variance of stratum mean residuals (proxy for BS)

If `Z` is specified, returns one row per treatment arm.

`stratification` Character string echoing the stratification variable name.

`evaluate_stratification_precoding` *Evaluate a candidate stratification strategy using pre-coding proxies*

Description

Since we don't have residuals, we use proxies:

- Variance of Yhat within strata (lower = more homogeneous predictions)
- Spread of stratum means (higher = better potential for bias correction)
- Balance of stratum sizes

Usage

```
evaluate_stratification_precoding(data, strata = NULL, Z = "Z")
```

Arguments

`data`	Data frame with Yhat column
`strata`	Stratification vector
`Z`	Character string specifying the name of the treatment indicator or `NULL` to ignore treatment groups.

Value

Data frame with proxy quality metrics

`generate_observed_sample` *Given a fully observed population, sample and generate observed outcomes, given possibly stratification, etc. This method would be used for simulation.*

Description

Given a fully observed population, sample and generate observed outcomes, given possibly stratification, etc. This method would be used for simulation.

Usage

```
generate_observed_sample(
  dat,
  n_code,
  Z = NULL,
  strata = NULL,
  by_tx = FALSE,
  seed = NULL
)
```

Arguments

n_code	Number to code. If single number, will roughly split across all strata under stratified proportional to the strata sizes. If a vector of length strata, will code that many in each strata (ignoring treatment assignment unless by_tx=TRUE). For srs, will sample ignoring treatment assignment unless by_tx = TRUE
strata	Name of stratification to use. If null will do SRS.

`generate_population` *Generate a population with strata and outcomes and surrogate/predicted outcomes that depend on strata.*

Description

These strata are generated post-treatment, in that we generate n_strat strata within the treated and control units separately, and then generate outcomes.

Usage

```
generate_population(
  N,
  n_strat,
  strata_config = c("balanced-exact", "balanced-approx", "unbalanced"),
  tau = 0,
  bias = c("none", "small", "moderate", "large", "extreme-contrast"),
  residual_var = c("homogeneous", "heterogeneous", "extreme-contrast"),
  sd_y = 3,
```

```
  R2_target = 0.5,
  no_tx = FALSE,
  rescale_v = TRUE,
  seed = NULL,
  oracle = TRUE
)
```

Arguments

no_tx	If TRUE, do not generate treatment assignment (Z) and set all outcomes to control potential outcome.

---

generate_sample_index *Given a full population, generate a vector of indicators for who is sampled.*

Description

Given a full population, generate a vector of indicators for who is sampled.

Usage

```
generate_sample_index(
  dat,
  n_code,
  Z = NULL,
  strata = NULL,
  p_tx = 0.5,
  seed = NULL
)
```

Arguments

n_code	Number to code. If single number, will roughly split across all strata under stratified proportional to the strata sizes. If a vector of length strata, will code that many in each strata (ignoring treatment assignment unless tx col passed). If tx col passed, then it is best to make sure the passed strata all are all treatment or all control, although it will interact strata with treatment indicator, and then order the strata as all control strata followed by all treatment strata. So if there is strata 1,2,3, you would get six strata of 1-0,2-0,3-0, and then 1-1,2-1,3-1. You would then need to pass a vector of length 6 for n_code (or a single number for total sample size). For SRS, will sample ignoring treatment assignment unless treatment column passed, and then either make two strata (a co and tx strata) and spread n_code across it, or take a vector of length 2 for n_code for control, then treatment.
Z	If NULL, sample ignoring treatment. If not null, sample within substrata defined by interacting strata with treatment.
strata	Name of stratification to use. If null will do SRS (i.e., treat all units as a single strata).
p_tx	Proportion of sample to allocate to treatment arm, if treatment arm is defined and n_code doesn't specify exact amounts for each strata. If NULL, then will allocation proportional to proportion actually treated in each strata.

`make_power_table` *Power curve table and plot for stratified vs SRS sampling*

Description

Build the power-curve table (via `make_power_table()`) and/or a ggplot (via `plot_power_curve()`) comparing stratified sampling to SRS, given population parameters.

Usage

```
make_power_table(N, p, var_Y0, R2_ML, R2_strat, h_min = 0.05)

plot_power_curve(N, p, var_Y0, R2_ML, R2_strat, h_min = 0.05)
```

Arguments

`N`	The total population size
`p`	The proportion of units assigned to treatment
`var_Y0`	The variance of the outcome in the control group
`R2_ML`	The pseudo R-squared of the model's surrogate outcomes for predicting actual outcomes.
`R2_strat`	The proportion of the residual variance explained by the stratification variable.
`h_min`	Minimum proportion of documents coded to include in the plot/table.

Details

You can estimate parameters from pilot data with `estimate_R2()`.

Value

`make_power_table()` returns a tibble with columns `h`, `SE`, and `MDES`.

`plot_power_curve()` returns a ggplot object with power curves. The underlying data table is stored as an attribute `"table"`.

See Also

`calc_nominal_var()` for calculating for a single scenario and fixed h, and `estimate_R2()` for estimating design parameterd.

plot_decomposition_comparison *Plot Oracle Decomposition Comparison*

Description

Creates a horizontal bar chart comparing the variance reduction (BS - WS) across candidate stratifications, with bars colored by the relative contribution of BS vs WS.

Usage

```
plot_decomposition_comparison(
  decomposition_result,
  top_n = NULL,
  show_components = FALSE
)
```

Arguments

decomposition_result	Output from compare_decomposition_oracle.
top_n	Integer specifying how many top stratifications to display. Default is NULL (show all).
show_components	Logical. If TRUE, shows separate bars for BS and WS. If FALSE (default), shows only BS - WS as a single bar.

Value

A ggplot object.

plot_strata_residuals *Create visualization of residual structure by strata*

Description

Create visualization of residual structure by strata

Usage

```
plot_strata_residuals(dat, strata)
```

Arguments

dat	Data frame with Yobs, Yhat, Z, strata columns
strata	Stratification variable

Value

ggplot object

Index

calc_nominal_var, 2
calc_nominal_var(), *10*, *17*
calc_optimal_allocation_oracle, 2
calc_sampling_weights, 3
compare_decomposition_oracle, 3, *18*
compare_oracle_vs_precoding, 5
compare_stratifications, *5, 6*, 6
compare_stratifications_precoding, *5, 6*, 6
compute_variance_decomposition, *4*, 7
create_stratification_candidates, 8

describe_population, 9

est_ma_srs, 10
est_ma_strat, 11
est_oracle, 12
est_subset, 12
estimate_R2, 10
estimate_R2(), *17*
evaluate_stratification, *6*, *8*, 13
evaluate_stratification_precoding, 14

generate_observed_sample, 15
generate_population, 15
generate_sample_index, 16

make_power_table, 17
make_power_table(), *10*

plot_decomposition_comparison, 18
plot_power_curve *(*make_power_table*)*, 17
plot_power_curve(), *10*
plot_strata_residuals, *8*, 18